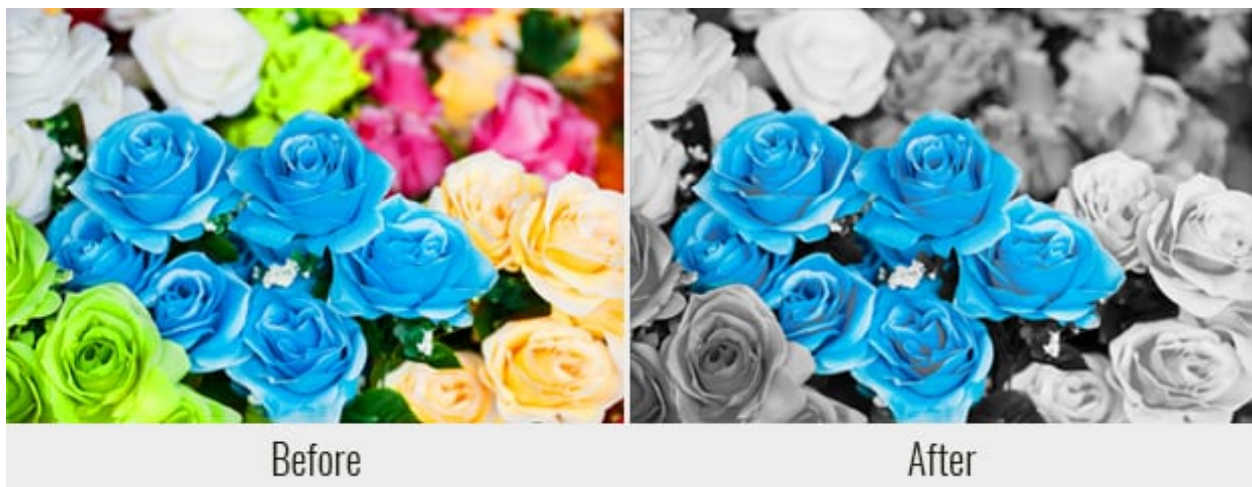


ECE 114 – Final Project

Topic: Image Transformation

TA: Varun Khatri

Let's have some fun with images! Almost every one of us have been using Instagram for the past few years and we all love uploading pretty pictures with different filters and effects (native and from Photoshop), which make our posts even more attractive. One very popular effect that people use is the one in which a picture is made Black & White except for one color. Something like this:



(Source: <https://enviragallery.com/how-to-make-a-photo-bw-except-one-color-in-photoshop/>)

In this project, we will use C/C++ to do the same processing a 24-bit BMP uncompressed image file. Before we get into the specifics of the project, let's try to understand how images are represented on a computer.

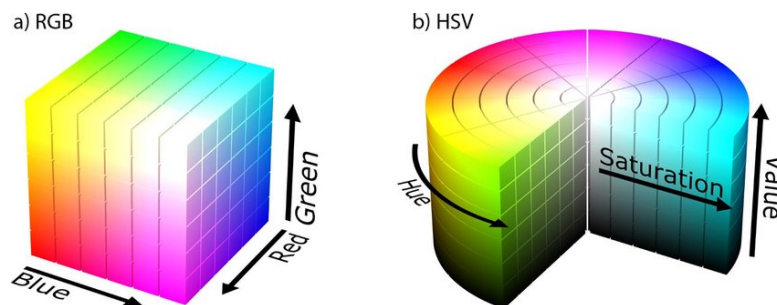
The basic building block of an image is a pixel. You can think of pixel as a small, colored square and a grid of these pixels form an image. A toy example of this would be this 9-pixel square with really thick edges, where pixel 5 is white and has a value 1 and all others are black and have the value 0.

1 (0)	2 (0)	3 (0)
4 (0)	5 (1)	6 (0)
7 (0)	8 (0)	9 (0)

Now, a BMP file, or a bitmap file, is a Windows file format in which the data in an image is organized in this same format. Formats such as JPEG and PNG, that you must be more familiar with, generally have lossy or lossless image compression and reading the bits from these files will not give us a “map” of the image. Since it’s not straightforward to deal with these files, for this project, we will be working only 24-bit uncompressed BMP files.

Why 24-bit? Well, that’s how much storage space is required by each pixel in a colored image. A very common way to represent color is by using the RGB model, in which the colors red, green and blue are added together in different ratios to form a new color. A BMP file uses the RGB model, where a pixel uses 24 bits to store a color, with red, green and blue using 8 bits each, taking a value between 0 and 255 (or 0x00 and 0xff in hex). So, a pure red would be 0xff, 0x00 and 0x00, whereas a pure green will be 0x00, 0xff, 0x00.

Notice how the RGB model will not be useful for our application because it’s tough to determine the exact ratio which represents a color. For example, a pixel in the flower image might have high blue content but might also have high red content and might not belong to the flowers which we want to leave untouched. Hence, we need to use another model. An alternate form of representation of color is the hue, saturation, value or the HSV model. Here, a particular angle (or hue) gives us the entire range of shades of a color and a particular color can be isolated by simply making saturation for other colors zero. A pictorial representation¹ of the two models is given below.



Before we start the implementation, there are a couple of things that we need to know.

We know that a bitmap file stores information pixel by pixel in a linear fashion. But that’s just the image data; what about the meta-data? Well, in a bitmap image, that is called Bitmap file header and is stored at the beginning 54 bytes of the file. This consists information like file type, size, dimensions, etc. Microsoft created their own structs to store this information. You can read more about it here:

<https://msdn.microsoft.com/en-us/library/cc230309.aspx>

[https://msdn.microsoft.com/en-us/library/dd183374\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/dd183374(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/dd183376\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/dd183376(v=vs.85).aspx)

¹ Popov, Vencislav & Ostarek, Markus & Tenison, Caitlin. (2018). Practices and pitfalls in inferring neural representations. *NeuroImage*. 174. 10.1016/j.neuroimage.2018.03.041.

Lastly, one additional part of a bitmap file data is 'padding'. As it turns out, total number of bytes allocated for a row of pixels in an image has to be a multiple of 4. For example, if we go back to the 9-pixel square, each row has 3 pixels, i.e, $3 \times 3 = 9$ bytes, which is not a multiple of 4. Hence, following these 9 bytes, there will be 3 bytes of zero-padding, which is essentially 24 zeros added to the back of the row data. The hex representation would be:

(0x00)	(0x00)	(0x00)	(0x00)
(0x00)	(0x00)	(0x00)	(0x00)
(0x00)	(0x00)	(0x00)	(0x00)
(0x00)	(0xff)	(0x00)	(0x00)
(0x00)	(0xff)	(0x00)	(0x00)
(0x00)	(0xff)	(0x00)	(0x00)
(0x00)	(0x00)	(0x00)	(0x00)
(0x00)	(0x00)	(0x00)	(0x00)
(0x00)	(0x00)	(0x00)	(0x00)

Now that we know everything about bitmaps and images in general, let's start coding!

Follow these steps outlined below to complete the program

1. Open 'bmp.h' provided in the package and read carefully. You'll find two functions `rgb2hsv` and `hsv2rgb` incomplete. Complete them according to definitions provided in the following links:
 - <https://www.rapidtables.com/convert/color/rgb-to-hsv.html>
 - <https://www.rapidtables.com/convert/color/hsv-to-rgb.html>
2. Create a .c file to write your main code and include the `bmp.h` file in it. If you're using C++, you may find 'extern "C"' expression useful.
3. Use command line arguments indicate input and output file names. Make sure you include statements for validation of file names and file pointers. Once the input file is open, read the bitmap file header and validate the following things:
 - File type should be BMP.
 - Offset to image data should be 54
 - Size of bitmap info header is 40 bytes.
 - Bit count per pixel is 24
 - Compression is off

Hint: You may find `fread`, `fwrite`, `fseek` helpful.
4. Determine whether 'padding' is required and start reading the file pixel by pixel. Convert values from RGB to HSV and change the color pixel to a grey pixel, if it doesn't match the color 'red'. Convert the values back to RGB and write the pixel into the output file. Move cursor (seek) beyond padding while reading the input file. Update the output file with padding, if required. Close the files.

Hint: Red values typically are between -20- and 20-degrees hue value. The conversion to grey scale can be done by making saturation zero.
5. Use the provided 'red_tele.bmp' to test your code.

You can try your own images with the software. Just make sure they are 24-bit uncompressed BMP files. Enjoy!

Important Note: If you are a Windows user, chances are that the code might not give you correct results. If that is the case, you may use CS50 IDE, a cloud-based development environment running on Linux, developed by Harvard. It looks very similar to VS Code and is easy to you.

To use CS50 IDE, you need to have an account on edX. If you don't have one, you can create one using your university email. Once the account is created, you can access it on this link <https://legacy.cs50.io>. You can check out a brief video explaining the full features on the IDE. (<https://www.youtube.com/watch?v=tEjbe85aZT4>)